
MP Online • Internship Project • 2025-26

GrievanceHub

Digital Grievance Redressal System

Design Document (DDD)

Architecture, Database Design & Application Flow

Prepared by: **Shantanu Shrivastava**

Date: April 2026

MP Online Internship Project

1. System Architecture

GrievanceHub follows a three-tier client-server architecture:

- Presentation Layer: React + Tailwind CSS SPA running on Vite dev server
- Application Layer: FastAPI backend exposing RESTful APIs via Uvicorn
- Data Layer: MySQL relational database accessed via SQLAlchemy ORM

1.1 Architecture Overview

Component	Technology	Port	Description
Frontend (Client)	React + Vite	5173	SPA in browser, communicates via Axios HTTP calls
Backend (Server)	FastAPI + Uvicorn	8000	REST API server handling all business logic
Database	MySQL 8.0	3306	Stores users, grievances, and ticket data
Auth Mechanism	JWT (python-jose)	N/A	Stateless token-based authentication

1.2 Backend Module Structure

Module	Folder	Responsibility
Models	app/models/	SQLAlchemy ORM classes mapping to DB tables
Schemas	app/schemas/	Pydantic models for request/response validation
Routers	app/routers/	API endpoint definitions and route handlers
Services	app/services/	Business logic separated from route handlers
Core	app/core/	Config, JWT security, dependencies (auth guards)
Database	app/database/	SQLAlchemy engine and session management
Utils	app/utils/	Ticket ID generator and helper functions

2. API Design

Method	Endpoint	Auth	Description
POST	/api/auth/register	None	Register a new citizen user
POST	/api/auth/login	None	Login and receive JWT token
POST	/api/grievances/submit	None	Submit grievance anonymously
POST	/api/grievances/submit/logged-in	JWT (Citizen)	Submit grievance as logged-in user
GET	/api/grievances/my	JWT (Citizen)	Get current user's grievances
GET	/api/tickets/{ticket_id}	None	Track grievance by ticket ID
GET	/api/admin/grievances	JWT (Admin)	Get all grievances with optional status filter
PATCH	/api/admin/grievances/{id}	JWT (Admin)	Update grievance status and remarks
GET	/api/admin/stats	JWT (Admin)	Get dashboard stats

3. Database Design

3.1 Users Table

Column	Data Type	Constraints	Description
id	INT	PK, AUTO INCREMENT	Unique user identifier
name	VARCHAR(100)	NOT NULL	Full name of the user
email	VARCHAR(150)	UNIQUE, NOT NULL	Login email address
password	VARCHAR(255)	NOT NULL	Bcrypt hashed password
phone	VARCHAR(15)	NULLABLE	Optional contact number
role	ENUM	DEFAULT 'citizen'	User role: citizen or admin
created_at	DATETIME	DEFAULT NOW()	Account creation timestamp

3.2 Grievances Table

Column	Data Type	Constraints	Description
id	INT	PK, AUTO INCREMENT	Unique grievance identifier
ticket_id	VARCHAR(20)	UNIQUE, NOT NULL	Auto-generated ticket (GH-YYYYMMDD-XXXXXX)
title	VARCHAR(200)	NOT NULL	Short title of the grievance
description	TEXT	NOT NULL	Detailed description of the complaint
category	VARCHAR(50)	NOT NULL	Grievance category (default: General)
priority	ENUM	DEFAULT 'low'	Priority: low, medium, high
status	ENUM	DEFAULT 'open'	Status: open, in_progress, resolved, closed
user_id	INT	FK (nullable)	References users.id; NULL for anonymous
submitter_name	VARCHAR(100)	NULLABLE	Name for anonymous submissions
submitter_email	VARCHAR(150)	NULLABLE	Email for anonymous submissions
admin_remarks	TEXT	NULLABLE	Resolution notes added by admin
created_at	DATETIME	DEFAULT NOW()	Submission timestamp
updated_at	DATETIME	AUTO UPDATE	Last modification timestamp

4. Application Flow

4.1 Grievance Submission Flow

Step	Actor	Action
1	Citizen	Opens GrievanceHub in browser, clicks Submit Grievance
2	Citizen	Chooses Anonymous or Logged-in submission
3	System	Shows form with name/email fields for anonymous users
4	Citizen	Fills title + description and submits form
5	System	Validates input via Pydantic schemas on backend

Step	Actor	Action
6	System	Generates unique ticket ID (GH-YYYYMMDD-XXXXXX)
7	System	Saves grievance to MySQL with status = open
8	System	Returns ticket ID to citizen for future tracking

4.2 Admin Resolution Flow

Step	Actor	Action
1	Admin	Logs in with admin credentials, JWT issued with role=admin
2	Admin	Views dashboard: Total, Open, In Progress, Resolved stats
3	Admin	Filters grievances by status using filter buttons
4	Admin	Selects grievance and updates status + adds remarks
5	System	Updates grievance record in DB and refreshes table

5. Security Design

- JWT tokens issued on login with 60-minute expiry via python-jose
- Admin routes protected by get_admin_user dependency (verifies role=admin in token)
- Passwords hashed using bcrypt via passlib before storing in DB
- CORS restricted to http://localhost:5173 (frontend origin only)
- Pydantic v2 validates all incoming request data automatically